

Universidad Nacional de Río Cuarto
Facultad de Ciencias Exactas, Físico-Químicas y Naturales
Departamento de Computación
Licenciatura en Ciencias de la Computación

Bases de Datos II

Trabajo Práctico N.º 1 Repaso de SQL

Alumno: Tomás Rodeghiero
Año: 2026

Índice

1. Introducción	2
2. Ejercicio 1: Definición de las bases de datos	2
2.1. Inciso a) Clientes, productos, ítems y facturas	2
2.2. Inciso b) Vehículos, personas, parquímetros y estacionamiento	5
2.3. Inciso c) Clientes, automóviles, talleres y accidentes	8
3. Ejercicio 2: Carga de datos	12
3.1. Inciso a) – MySQL/PostgreSQL	12
3.2. Inciso b) – MySQL	13
3.3. Inciso b) – Oracle	14
3.4. Inciso c) – MySQL/PostgreSQL	15
4. Ejercicio 3: Consultas SQL y álgebra relacional	15
4.1. a) Clientes sin ventas (base del inciso 1.a)	16
4.2. b) Vehículos que usaron el parquímetro 9 (base del inciso 1.b)	17
4.3. c) Clientes con más de 3 accidentes (base del inciso 1.c)	17
4.4. d) Máximo y mínimo monto de factura por cliente (base del inciso 1.a)	18
4.5. e) Tres consultas en álgebra relacional	19
5. Conclusión	21

1. Introducción

Este trabajo práctico es un repaso integral de SQL. Se divide en tres ejercicios: primero definir el esquema de tres bases de datos identificando claves primarias, secundarias (**UNIQUE**), foráneas, integridad referencial y restricciones de dominio; segundo, cargar datos coherentes; y tercero, resolver consultas SQL y plantear consultas adicionales en álgebra relacional.

La resolución usa los motores pedidos por la cátedra: MySQL 8 o superior, PostgreSQL 14 o superior y Oracle XE 10g/11g. Cuando un motor no soporta una construcción del SQL estándar (por ejemplo **CREATE DOMAIN** en MySQL u Oracle), se documenta la alternativa empleada para emularla.

Para evitar colisiones de nombres entre tablas, cada base se aísla en un esquema o base propia: **practico1a**, **practico1b** y **practico1c**. Los datos cargados en el Ejercicio 2 están pensados para que las consultas del Ejercicio 3 devuelvan resultados significativos.

Cada ejercicio sigue siempre la misma estructura: **Consigna** (texto literal del enunciado), **Qué se pide / idea** (lectura natural del problema), **Solución en SQL** (código por motor) y **Por qué funciona** (lógica de la resolución).

2. Ejercicio 1: Definición de las bases de datos

Consigna

Crear las bases de Datos e Identificar las claves primarias, secundarias (cláusula **UNIQUE**), Claves foráneas, cláusulas de integridad referencial (**Foreign key**), restricciones de dominio; correspondientes a los ejercicios:

Qué se pide / idea

Es un ejercicio de DDL puro: por cada esquema hay que escribir el **CREATE TABLE** con **PRIMARY KEY**, **UNIQUE**, **CHECK** y **FOREIGN KEY**, y elegir las acciones referenciales (**ON DELETE CASCADE** o **RESTRICT**) según lo que pide cada inciso. Cuando el motor no expone **CREATE DOMAIN**, los dominios se emulan con **CHECK** sobre la columna o con tipos enumerados.

2.1. Inciso a) Clientes, productos, ítems y facturas

Consigna

Cliente (nro_cliente, apellido, nombre, dirección, teléfono)
Producto (cod_producto, descripción, precio, stock_minimo, stock_maximo, cantidad)
ItemFactura (cod_producto, nro_factura, cantidad, precio)
Factura (nro_factura, nro_cliente, fecha, monto)

Considerar el precio no puede ser 0 ni negativo; el Stock mínimo no puede ser mayor que el Stock máximo. Tener en cuenta al borrar un cliente debe eliminarse toda la información de facturas realizadas para el mismo; y al eliminar un producto, no permitir hacerlo si hay alguna factura en la que fue utilizado.

Nota: Resolver utilizando los motores de MySQL y Postgres.

Qué se pide / idea

Hay cuatro tablas con una jerarquía clara: **cliente** y **producto** son las entidades; **factura** es 1:N respecto de **cliente**; e **item_factura** es la tabla intermedia con clave primaria compuesta (cod_producto, nro_factura). La parte interesante es traducir las reglas en lenguaje natural

a SQL: “precio mayor a 0” es un CHECK; “al borrar un cliente, eliminar sus facturas” es ON DELETE CASCADE; “no dejar borrar un producto facturado” es ON DELETE RESTRICT.

Solución en SQL

MySQL 8+:

```
CREATE DATABASE IF NOT EXISTS practicola_mysql;
USE practicola_mysql;

CREATE TABLE cliente (
    nro_cliente INT PRIMARY KEY,
    apellido VARCHAR(60) NOT NULL,
    nombre VARCHAR(60) NOT NULL,
    direccion VARCHAR(120) NOT NULL,
    telefono VARCHAR(20) NOT NULL,
    CONSTRAINT uq_cliente_telefono UNIQUE (telefono)
);

CREATE TABLE producto (
    cod_producto INT PRIMARY KEY,
    descripcion VARCHAR(120) NOT NULL,
    precio DECIMAL(10,2) NOT NULL,
    stock_minimo INT NOT NULL,
    stock_maximo INT NOT NULL,
    cantidad INT NOT NULL,
    CONSTRAINT ck_producto_precio CHECK (precio > 0),
    CONSTRAINT ck_producto_stock_rango CHECK (stock_minimo >= 0 AND stock_maximo >= 0),
    CONSTRAINT ck_producto_stock_logico CHECK (stock_minimo <= stock_maximo),
    CONSTRAINT ck_producto_cantidad CHECK (cantidad >= 0)
);

CREATE TABLE factura (
    nro_factura INT PRIMARY KEY,
    nro_cliente INT NOT NULL,
    fecha DATE NOT NULL,
    monto DECIMAL(12,2) NOT NULL,
    CONSTRAINT ck_factura_monto CHECK (monto > 0),
    CONSTRAINT fk_factura_cliente
        FOREIGN KEY (nro_cliente)
        REFERENCES cliente(nro_cliente)
        ON DELETE CASCADE
);

CREATE TABLE item_factura (
    cod_producto INT NOT NULL,
    nro_factura INT NOT NULL,
    cantidad INT NOT NULL,
    precio DECIMAL(10,2) NOT NULL,
    CONSTRAINT pk_item_factura PRIMARY KEY (cod_producto, nro_factura),
    CONSTRAINT ck_item_cantidad CHECK (cantidad > 0),
    CONSTRAINT ck_item_precio CHECK (precio > 0),
    CONSTRAINT fk_item_producto
        FOREIGN KEY (cod_producto)
        REFERENCES producto(cod_producto)
        ON DELETE RESTRICT,
    CONSTRAINT fk_item_factura
        FOREIGN KEY (nro_factura)
        REFERENCES factura(nro_factura)
        ON DELETE CASCADE
);
```

PostgreSQL 14+:

```
CREATE SCHEMA IF NOT EXISTS practico1a;
SET search_path TO practico1a;

CREATE TABLE cliente (
    nro_cliente INTEGER PRIMARY KEY,
    apellido VARCHAR(60) NOT NULL,
    nombre VARCHAR(60) NOT NULL,
    direccion VARCHAR(120) NOT NULL,
    telefono VARCHAR(20) NOT NULL,
    CONSTRAINT uq_cliente_telefono UNIQUE (telefono)
);

CREATE TABLE producto (
    cod_producto INTEGER PRIMARY KEY,
    descripcion VARCHAR(120) NOT NULL,
    precio NUMERIC(10,2) NOT NULL,
    stock_minimo INTEGER NOT NULL,
    stock_maximo INTEGER NOT NULL,
    cantidad INTEGER NOT NULL,
    CONSTRAINT ck_producto_precio CHECK (precio > 0),
    CONSTRAINT ck_producto_stock_rango CHECK (stock_minimo >= 0 AND stock_maximo >= 0),
    CONSTRAINT ck_producto_stock_logico CHECK (stock_minimo <= stock_maximo),
    CONSTRAINT ck_producto_cantidad CHECK (cantidad >= 0)
);

CREATE TABLE factura (
    nro_factura INTEGER PRIMARY KEY,
    nro_cliente INTEGER NOT NULL,
    fecha DATE NOT NULL,
    monto NUMERIC(12,2) NOT NULL,
    CONSTRAINT ck_factura_monto CHECK (monto > 0),
    CONSTRAINT fk_factura_cliente
        FOREIGN KEY (nro_cliente)
        REFERENCES cliente(nro_cliente)
        ON DELETE CASCADE
);

CREATE TABLE item_factura (
    cod_producto INTEGER NOT NULL,
    nro_factura INTEGER NOT NULL,
    cantidad INTEGER NOT NULL,
    precio NUMERIC(10,2) NOT NULL,
    CONSTRAINT pk_item_factura PRIMARY KEY (cod_producto, nro_factura),
    CONSTRAINT ck_item_cantidad CHECK (cantidad > 0),
    CONSTRAINT ck_item_precio CHECK (precio > 0),
    CONSTRAINT fk_item_producto
        FOREIGN KEY (cod_producto)
        REFERENCES producto(cod_producto)
        ON DELETE RESTRICT,
    CONSTRAINT fk_item_factura
        FOREIGN KEY (nro_factura)
        REFERENCES factura(nro_factura)
        ON DELETE CASCADE
);
```

Por qué funciona

- Las claves primarias son las del enunciado (nro_cliente, cod_producto, nro_factura) y la compuesta (cod_producto, nro_factura) en item_factura evita que un mismo producto figure dos veces en la misma factura.

- Se agrega UNIQUE (**telefono**) como clave secundaria razonable: en este dominio dos clientes distintos no comparten teléfono.
- Para cumplir “al borrar un cliente, eliminar sus facturas”, la FK **factura.nro_cliente** usa ON DELETE CASCADE. Para evitar ítems huérfanos al borrar una factura, **item_factura.nro_factura** también cascadea.
- Para que un producto facturado no pueda borrarse, **item_factura.cod_producto** usa ON DELETE RESTRICT, que aborta el DELETE si quedan referencias.
- Los dominios pedidos se expresan con CHECK: **precio >0**, **stock_minimo <= stock_maximo** y stocks no negativos.

2.2. Inciso b) Vehículos, personas, parquímetros y estacionamiento

Consigna

Vehiculo (#patente, marca, modelo, color, saldoActual)
 Persona (dni, nombreYApellido, direccion)
 Dueño (#patente, dni)
 Estacionamiento (#estacionamiento, #patente, #parquímetro, fecha, saldoInicio, saldoFinal, horaEntrada, horaSalida)
 Parquímetro (#parquímetro, calle, altura)

Considerar que el número de calle no puede ser un número negativo ni superior a 5000; crear un dominio para el nombre y apellido de una persona que es un varchar de 45. Considerar que el dominio del atributo color de un vehículo es {gris, negro, azul}. No permitir eliminar un vehículo si este fue estacionado. El #estacionamiento debe ser autonumerado.

Nota: Resolver utilizando los motores de MySQL, Oracle.

Qué se pide / idea

Hay una relación N:N entre **persona** y **vehiculo** que se resuelve con la tabla puente **duenio**. La tabla central es **estacionamiento**, que referencia simultáneamente a un **vehiculo** y a un **parquimetro**. Lo más sutil es la diferencia entre motores: MySQL tiene ENUM y AUTO_INCREMENT, Oracle no, y hay que emular ambos con CHECK IN (...) y con SEQUENCE + trigger BEFORE INSERT.

Solución en SQL

MySQL 8+:

```
CREATE DATABASE IF NOT EXISTS practico1b_mysql;
USE practico1b_mysql;

CREATE TABLE persona (
  dni BIGINT PRIMARY KEY,
  nombre_y_apellido VARCHAR(45) NOT NULL,
  direccion VARCHAR(120) NOT NULL
);

CREATE TABLE vehiculo (
  patente VARCHAR(10) PRIMARY KEY,
  marca VARCHAR(40) NOT NULL,
  modelo INT NOT NULL,
  color ENUM('gris', 'negro', 'azul') NOT NULL,
  saldo_actual DECIMAL(10,2) NOT NULL DEFAULT 0,
  CONSTRAINT ck_vehiculo_saldo CHECK (saldo_actual >= 0)
);

CREATE TABLE parquimetro (
```

```

    id_parquimetro INT PRIMARY KEY,
    calle VARCHAR(80) NOT NULL,
    altura INT NOT NULL,
    CONSTRAINT uq_parquimetro_ubicacion UNIQUE (calle, altura),
    CONSTRAINT ck_parquimetro_altura CHECK (altura BETWEEN 0 AND 5000)
);

CREATE TABLE duenio (
    patente VARCHAR(10) NOT NULL,
    dni BIGINT NOT NULL,
    CONSTRAINT pk_duenio PRIMARY KEY (patente, dni),
    CONSTRAINT fk_duenio_vehiculo
        FOREIGN KEY (patente)
        REFERENCES vehiculo(patente)
        ON DELETE CASCADE,
    CONSTRAINT fk_duenio_persona
        FOREIGN KEY (dni)
        REFERENCES persona(dni)
        ON DELETE CASCADE
);

CREATE TABLE estacionamiento (
    id_estacionamiento INT AUTO_INCREMENT PRIMARY KEY,
    patente VARCHAR(10) NOT NULL,
    id_parquimetro INT NOT NULL,
    fecha DATE NOT NULL,
    saldo_inicio DECIMAL(10,2) NOT NULL,
    saldo_final DECIMAL(10,2) NOT NULL,
    hora_entrada TIME NOT NULL,
    hora_salida TIME,
    CONSTRAINT ck_estacionamiento_saldos CHECK (saldo_inicio >= 0 AND saldo_final >= 0),
    CONSTRAINT ck_estacionamiento_consumo CHECK (saldo_final <= saldo_inicio),
    CONSTRAINT ck_estacionamiento_horas CHECK (hora_salida IS NULL OR hora_salida >=
    hora_entrada),
    CONSTRAINT fk_estacionamiento_vehiculo
        FOREIGN KEY (patente)
        REFERENCES vehiculo(patente),
    CONSTRAINT fk_estacionamiento_parquimetro
        FOREIGN KEY (id_parquimetro)
        REFERENCES parquemetro(id_parquimetro)
);

```

Oracle XE 10g/11g: primero, como administrador, se crea el usuario que actúa como esquema:

```

CREATE USER practico1b IDENTIFIED BY practico1b;
GRANT CREATE SESSION, CREATE TABLE, CREATE SEQUENCE, CREATE TRIGGER TO practico1b;
ALTER USER practico1b QUOTA UNLIMITED ON USERS;

```

Luego, ya como practico1b:

```

CREATE TABLE persona (
    dni NUMBER(10) PRIMARY KEY,
    nombre_y_apellido VARCHAR2(45 CHAR) NOT NULL,
    direccion VARCHAR2(120 CHAR) NOT NULL
);

CREATE TABLE vehiculo (
    patente VARCHAR2(10 CHAR) PRIMARY KEY,
    marca VARCHAR2(40 CHAR) NOT NULL,
    modelo NUMBER(4) NOT NULL,
    color VARCHAR2(10 CHAR) NOT NULL,
    saldo_actual NUMBER(10,2) DEFAULT 0 NOT NULL,

```

```

CONSTRAINT ck_vehiculo_color CHECK (LOWER(color) IN ('gris', 'negro', 'azul')),
CONSTRAINT ck_vehiculo_saldo CHECK (saldo_actual >= 0)
);

CREATE TABLE parquemetro (
    id_parquemetro NUMBER(10) PRIMARY KEY,
    calle VARCHAR2(80 CHAR) NOT NULL,
    altura NUMBER(4) NOT NULL,
    CONSTRAINT uq_parquemetro_ubicacion UNIQUE (calle, altura),
    CONSTRAINT ck_parquemetro_altura CHECK (altura BETWEEN 0 AND 5000)
);

CREATE TABLE duenio (
    patente VARCHAR2(10 CHAR) NOT NULL,
    dni NUMBER(10) NOT NULL,
    CONSTRAINT pk_duenio PRIMARY KEY (patente, dni),
    CONSTRAINT fk_duenio_vehiculo
        FOREIGN KEY (patente)
        REFERENCES vehiculo(patente)
        ON DELETE CASCADE,
    CONSTRAINT fk_duenio_persona
        FOREIGN KEY (dni)
        REFERENCES persona(dni)
        ON DELETE CASCADE
);

CREATE TABLE estacionamiento (
    id_estacionamiento NUMBER(10) PRIMARY KEY,
    patente VARCHAR2(10 CHAR) NOT NULL,
    id_parquemetro NUMBER(10) NOT NULL,
    fecha DATE NOT NULL,
    saldo_inicio NUMBER(10,2) NOT NULL,
    saldo_final NUMBER(10,2) NOT NULL,
    hora_entrada TIMESTAMP NOT NULL,
    hora_salida TIMESTAMP,
    CONSTRAINT ck_estacionamiento_saldos CHECK (saldo_inicio >= 0 AND saldo_final >= 0),
    CONSTRAINT ck_estacionamiento_consumo CHECK (saldo_final <= saldo_inicio),
    CONSTRAINT ck_estacionamiento_horas CHECK (hora_salida IS NULL OR hora_salida >=
    hora_entrada),
    CONSTRAINT fk_estacionamiento_vehiculo
        FOREIGN KEY (patente)
        REFERENCES vehiculo(patente),
    CONSTRAINT fk_estacionamiento_parquemetro
        FOREIGN KEY (id_parquemetro)
        REFERENCES parquemetro(id_parquemetro)
);

CREATE SEQUENCE seq_estacionamiento START WITH 1 INCREMENT BY 1 NOCACHE;

CREATE OR REPLACE TRIGGER trg_estacionamiento_bi
BEFORE INSERT ON estacionamiento
FOR EACH ROW
WHEN (NEW.id_estacionamiento IS NULL)
BEGIN
    SELECT seq_estacionamiento.NEXTVAL
    INTO :NEW.id_estacionamiento
    FROM dual;
END;
/

```

Por qué funciona

- **duenio** resuelve la N:N entre vehículos y personas con PK compuesta (**patente**, **dni**); las dos FK usan **CASCADE** para que al borrar una persona o un vehículo desaparezcan las titularidades asociadas, sin tocar el resto del modelo.
- Para impedir borrar un vehículo que fue estacionado, la FK **estacionamiento.patente** se deja con el comportamiento por defecto (**NO ACTION/RESTRICT**); si hay estacionamientos vinculados, el motor aborta el **DELETE**.
- El dominio del color usa **ENUM** en MySQL y **CHECK (LOWER(color) IN (...))** en Oracle. El dominio del nombre y apellido (**VARCHAR(45)**) se aplica directamente sobre la columna, ya que ni MySQL ni Oracle soportan **CREATE DOMAIN**.
- La autonumeración del **id_estacionamiento** es **AUTO_INCREMENT** en MySQL; en Oracle 10/11 XE se combina una **SEQUENCE** con un trigger **BEFORE INSERT** que asigna el siguiente valor solo cuando el insert no lo proporciona (en Oracle 12+ existe **GENERATED AS IDENTITY**, pero la cátedra apunta a 10/11).
- **UNIQUE (calle, altura)** en **parquimetro** evita dos parquímetros distintos con la misma ubicación física.

2.3. Inciso c) Clientes, automóviles, talleres y accidentes

Consigna

Dado el siguiente esquema:

```

Cliente (DNI, nombre, apellido, dirección, tarifa)
Automovil (patente, marca, modelo, dni, nro_categoria)
Categoria (nro_categoria, tasa)
Taller (nro_taller, nombre, dirección)
Accidente (nro_accidente, DNI, patente, nro_taller, fecha, costo)

DNI clave foránea que referencia DNI de Cliente
nro_taller clave foránea que referencia nro_taller de Taller
patente clave foránea que referencia patente de Automóvil

```

Considerar que el modelo de un automóvil es un entero positivo entre 1990 y 2015. El atributo marca de un automóvil puede ser {FIAT, RENAULT, FORD}. El atributo tarifa es un entero positivo menos a 10000. Tener en cuenta al borrar un cliente debe eliminarse toda la información de los accidentes que lo involucran.

Nota: Resolver utilizando los motores de PostgreSQL y ORACLE.

Qué se pide / idea

El enunciado declara las FK “visibles” de **accidente**, pero **automovil.dni** y **automovil.nro_categoria** también necesitan ser FK para mantener integridad referencial. La regla “al borrar un cliente, borrar sus accidentes” se modela cascadeando desde **cliente** hacia **automovil** y **accidente**, de modo que un solo **DELETE** sobre **cliente** dispare la limpieza completa.

Solución en SQL

Aunque la cátedra solo pide PostgreSQL y Oracle para este inciso, el DDL se traduce sin pérdida a MySQL 8+: todas las restricciones (**CHECK** sobre rangos, dominios por enumeración con **IN (...)** y FK con **ON DELETE CASCADE**) están soportadas. Por eso se incluye también la versión MySQL.

MySQL 8+:

```

CREATE DATABASE IF NOT EXISTS practico1c_mysql;
USE practico1c_mysql;

```

```

CREATE TABLE cliente (
    dni BIGINT PRIMARY KEY,
    nombre VARCHAR(60) NOT NULL,
    apellido VARCHAR(60) NOT NULL,
    direccion VARCHAR(120) NOT NULL,
    tarifa INT NOT NULL,
    CONSTRAINT ck_cliente_tarifa CHECK (tarifa > 0 AND tarifa < 10000)
);

CREATE TABLE categoria (
    nro_categoria INT PRIMARY KEY,
    tasa DECIMAL(6,2) NOT NULL,
    CONSTRAINT ck_categoria_tasa CHECK (tasa > 0)
);

CREATE TABLE taller (
    nro_taller INT PRIMARY KEY,
    nombre VARCHAR(80) NOT NULL,
    direccion VARCHAR(120) NOT NULL
);

CREATE TABLE automovil (
    patente VARCHAR(10) PRIMARY KEY,
    marca VARCHAR(10) NOT NULL,
    modelo INT NOT NULL,
    dni BIGINT NOT NULL,
    nro_categoria INT NOT NULL,
    CONSTRAINT ck_automovil_modelo CHECK (modelo BETWEEN 1990 AND 2015),
    CONSTRAINT ck_automovil_marca CHECK (marca IN ('FIAT', 'RENAULT', 'FORD')),
    CONSTRAINT fk_automovil_cliente
        FOREIGN KEY (dni)
        REFERENCES cliente(dni)
        ON DELETE CASCADE,
    CONSTRAINT fk_automovil_categoria
        FOREIGN KEY (nro_categoria)
        REFERENCES categoria(nro_categoria)
);

CREATE TABLE accidente (
    nro_accidente INT PRIMARY KEY,
    dni BIGINT NOT NULL,
    patente VARCHAR(10) NOT NULL,
    nro_taller INT NOT NULL,
    fecha DATE NOT NULL,
    costo DECIMAL(12,2) NOT NULL,
    CONSTRAINT ck_accidente_costo CHECK (costo > 0),
    CONSTRAINT fk_accidente_cliente
        FOREIGN KEY (dni)
        REFERENCES cliente(dni)
        ON DELETE CASCADE,
    CONSTRAINT fk_accidente_automovil
        FOREIGN KEY (patente)
        REFERENCES automovil(patente)
        ON DELETE CASCADE,
    CONSTRAINT fk_accidente_taller
        FOREIGN KEY (nro_taller)
        REFERENCES taller(nro_taller)
);

```

PostgreSQL 14+:

```
CREATE SCHEMA IF NOT EXISTS practico1c;
```

```
SET search_path TO practico1c;

CREATE TABLE cliente (
    dni BIGINT PRIMARY KEY,
    nombre VARCHAR(60) NOT NULL,
    apellido VARCHAR(60) NOT NULL,
    direccion VARCHAR(120) NOT NULL,
    tarifa INTEGER NOT NULL,
    CONSTRAINT ck_cliente_tarifa CHECK (tarifa > 0 AND tarifa < 10000)
);

CREATE TABLE categoria (
    nro_categoria INTEGER PRIMARY KEY,
    tasa NUMERIC(6,2) NOT NULL,
    CONSTRAINT ck_categoria_tasa CHECK (tasa > 0)
);

CREATE TABLE taller (
    nro_taller INTEGER PRIMARY KEY,
    nombre VARCHAR(80) NOT NULL,
    direccion VARCHAR(120) NOT NULL
);

CREATE TABLE automovil (
    patente VARCHAR(10) PRIMARY KEY,
    marca VARCHAR(10) NOT NULL,
    modelo INTEGER NOT NULL,
    dni BIGINT NOT NULL,
    nro_categoria INTEGER NOT NULL,
    CONSTRAINT ck_automovil_modelo CHECK (modelo BETWEEN 1990 AND 2015),
    CONSTRAINT ck_automovil_marca CHECK (marca IN ('FIAT', 'RENAULT', 'FORD')),
    CONSTRAINT fk_automovil_cliente
        FOREIGN KEY (dni)
        REFERENCES cliente(dni)
        ON DELETE CASCADE,
    CONSTRAINT fk_automovil_categoria
        FOREIGN KEY (nro_categoria)
        REFERENCES categoria(nro_categoria)
);

CREATE TABLE accidente (
    nro_accidente INTEGER PRIMARY KEY,
    dni BIGINT NOT NULL,
    patente VARCHAR(10) NOT NULL,
    nro_taller INTEGER NOT NULL,
    fecha DATE NOT NULL,
    costo NUMERIC(12,2) NOT NULL,
    CONSTRAINT ck_accidente_costo CHECK (costo > 0),
    CONSTRAINT fk_accidente_cliente
        FOREIGN KEY (dni)
        REFERENCES cliente(dni)
        ON DELETE CASCADE,
    CONSTRAINT fk_accidente_automovil
        FOREIGN KEY (patente)
        REFERENCES automovil(patente)
        ON DELETE CASCADE,
    CONSTRAINT fk_accidente_taller
        FOREIGN KEY (nro_taller)
        REFERENCES taller(nro_taller)
);
```

Oracle XE 10g/11g:

```
CREATE USER practico1c IDENTIFIED BY practico1c;
GRANT CREATE SESSION, CREATE TABLE TO practico1c;
ALTER USER practico1c QUOTA UNLIMITED ON USERS;
```

Y luego, ya como practico1c:

```
CREATE TABLE cliente (
    dni NUMBER(10) PRIMARY KEY,
    nombre VARCHAR2(60 CHAR) NOT NULL,
    apellido VARCHAR2(60 CHAR) NOT NULL,
    direccion VARCHAR2(120 CHAR) NOT NULL,
    tarifa NUMBER(5) NOT NULL,
    CONSTRAINT ck_cliente_tarifa CHECK (tarifa > 0 AND tarifa < 10000)
);

CREATE TABLE categoria (
    nro_categoria NUMBER(10) PRIMARY KEY,
    tasa NUMBER(6,2) NOT NULL,
    CONSTRAINT ck_categoria_tasa CHECK (tasa > 0)
);

CREATE TABLE taller (
    nro_taller NUMBER(10) PRIMARY KEY,
    nombre VARCHAR2(80 CHAR) NOT NULL,
    direccion VARCHAR2(120 CHAR) NOT NULL
);

CREATE TABLE automovil (
    patente VARCHAR2(10 CHAR) PRIMARY KEY,
    marca VARCHAR2(10 CHAR) NOT NULL,
    modelo NUMBER(4) NOT NULL,
    dni NUMBER(10) NOT NULL,
    nro_categoria NUMBER(10) NOT NULL,
    CONSTRAINT ck_automovil_modelo CHECK (modelo BETWEEN 1990 AND 2015),
    CONSTRAINT ck_automovil_marca CHECK (marca IN ('FIAT', 'RENAULT', 'FORD')),
    CONSTRAINT fk_automovil_cliente
        FOREIGN KEY (dni)
        REFERENCES cliente(dni)
        ON DELETE CASCADE,
    CONSTRAINT fk_automovil_categoria
        FOREIGN KEY (nro_categoria)
        REFERENCES categoria(nro_categoria)
);

CREATE TABLE accidente (
    nro_accidente NUMBER(10) PRIMARY KEY,
    dni NUMBER(10) NOT NULL,
    patente VARCHAR2(10 CHAR) NOT NULL,
    nro_taller NUMBER(10) NOT NULL,
    fecha DATE NOT NULL,
    costo NUMBER(12,2) NOT NULL,
    CONSTRAINT ck_accidente_costo CHECK (costo > 0),
    CONSTRAINT fk_accidente_cliente
        FOREIGN KEY (dni)
        REFERENCES cliente(dni)
        ON DELETE CASCADE,
    CONSTRAINT fk_accidente_automovil
        FOREIGN KEY (patente)
        REFERENCES automovil(patente)
        ON DELETE CASCADE,
    CONSTRAINT fk_accidente_taller
        FOREIGN KEY (nro_taller)
```

```
REFERENCES taller(nro_taller)
);
```

Por qué funciona

- `cliente` es la entidad raíz: la cascada `cliente` → `automovil` → `accidente` (más `cliente` → `accidente` directamente) garantiza que un único `DELETE FROM cliente WHERE dni = ...` elimine en limpio toda la información asociada, tal como pide el enunciado.
- `accidente.nro_taller` usa el comportamiento por defecto (`RESTRICT`): borrar un taller con accidentes registrados se prohíbe, porque la traza queda atada al taller.
- Los dominios se traducen en `CHECK: modelo BETWEEN 1990 AND 2015, marca IN ('FIAT', 'RENAULT', 'FORD')` y `tarifa > 0 AND tarifa < 10000` reflejan exactamente las restricciones del enunciado.
- `tarifa` se declara `INTEGER` porque el enunciado habla de “entero positivo”; el `CHECK` cubre el rango (0, 10000) excluyendo el cero.

3. Ejercicio 2: Carga de datos

Consigna

Utilizando SQL, agregar datos coherentes a las bases de datos creadas en los incisos del ejercicio anterior.

Qué se pide / idea

Es `INSERT` puro, pero los datos no son arbitrarios: tienen que respetar todas las restricciones del DDL (claves, dominios, integridad referencial) y dejar el estado preparado para que cada consulta del Ejercicio 3 devuelva un resultado significativo. Concretamente, en 1.a se carga un cliente sin facturas, en 1.b se reparten estacionamientos sobre el parquímetro 9, y en 1.c se carga un cliente con cuatro accidentes para que `HAVING COUNT(*) > 3` matchee.

3.1. Inciso a) – MySQL/PostgreSQL

Solución en SQL

```
INSERT INTO cliente (nro_cliente, apellido, nombre, direccion, telefono) VALUES
(1, 'Perez', 'Andres', 'San Martin 123', '3584010001'),
(2, 'Diaz', 'Bruno', 'Belgrano 450', '3584010002'),
(3, 'Gomez', 'Carlos', 'Mitre 980', '3584010003'),
(4, 'Luna', 'Diego', 'Constitucion 75', '3584010004');

INSERT INTO producto (cod_producto, descripcion, precio, stock_minimo, stock_maximo,
cantidad) VALUES
(101, 'Teclado mecanico', 15000.00, 5, 30, 12),
(102, 'Mouse optico', 8000.00, 10, 40, 25),
(103, 'Monitor 24', 90000.00, 2, 15, 6),
(104, 'Notebook 14', 450000.00, 1, 8, 3);

INSERT INTO factura (nro_factura, nro_cliente, fecha, monto) VALUES
(1001, 1, '2024-08-01', 106000.00),
(1002, 1, '2024-08-10', 450000.00),
(1003, 2, '2024-08-12', 46000.00),
(1004, 3, '2024-08-15', 98000.00);

INSERT INTO item_factura (cod_producto, nro_factura, cantidad, precio) VALUES
```

```
(103, 1001, 1, 90000.00),
(102, 1001, 2, 8000.00),
(104, 1002, 1, 450000.00),
(101, 1003, 2, 15000.00),
(102, 1003, 2, 8000.00),
(103, 1004, 1, 90000.00),
(102, 1004, 1, 8000.00);
```

Por qué funciona

- Diego Luna queda intencionalmente sin facturas, para que aparezca en la consulta de clientes sin ventas (3.a).
- Andrés Pérez tiene dos facturas con montos distintos, lo que hace que MAX y MIN por cliente devuelvan valores diferenciados (3.d).
- Los montos declarados en cada factura coinciden con la suma de **cantidad * precio** de sus ítems, así no aparece inconsistencia conceptual entre las dos tablas.

3.2. Inciso b) – MySQL

Solución en SQL

```
INSERT INTO persona (dni, nombre_y_apellido, direccion) VALUES
(30111222, 'Juan Perez', 'Laprida 120'),
(28999111, 'Mario Lopez', 'Lavalle 455'),
(33444555, 'Pedro Suarez', 'Mitre 900');

INSERT INTO vehiculo (patente, marca, modelo, color, saldo_actual) VALUES
('AA123BB', 'Renault', 2018, 'gris', 1500.00),
('AC456DD', 'Ford', 2020, 'negro', 800.00),
('AD789EE', 'Fiat', 2016, 'azul', 1200.00);

INSERT INTO parquemetro (id_parquemetro, calle, altura) VALUES
(9, 'Italia', 1200),
(10, 'Sarmiento', 845),
(11, 'San Martin', 2300);

INSERT INTO duenio (patente, dni) VALUES
('AA123BB', 30111222),
('AC456DD', 28999111),
('AD789EE', 33444555);

INSERT INTO estacionamiento
(patente, id_parquemetro, fecha, saldo_inicio, saldo_final, hora_entrada, hora_salida)
VALUES
('AA123BB', 9, '2024-08-02', 1500.00, 1350.00, '08:00:00', '09:30:00'),
('AC456DD', 9, '2024-08-02', 800.00, 650.00, '10:15:00', '11:45:00'),
('AD789EE', 10, '2024-08-03', 1200.00, 1000.00, '18:00:00', '19:40:00'),
('AA123BB', 11, '2024-08-04', 1350.00, 1250.00, '16:00:00', '16:50:00');
```

Por qué funciona

- Hay dos vehículos distintos (AA123BB y AC456DD) que pasaron por el parquímetro 9, así la consulta 3.b devuelve más de una fila.
- Todos los INSERT respetan el dominio del color (ENUM('gris', 'negro', 'azul')) y la restricción altura BETWEEN 0 AND 5000.
- No se insertan valores para id_estacionamiento: lo asigna el AUTO_INCREMENT.

3.3. Inciso b) – Oracle

En Oracle se omite `id_estacionamiento` en el INSERT: el trigger `trg_estacionamiento_bi` se encarga de tomar el siguiente valor de `seq_estacionamiento`.

Solución en SQL

```
INSERT INTO persona (dni, nombre_y_apellido, direccion)
VALUES (30111222, 'Juan Perez', 'Laprida 120');
INSERT INTO persona (dni, nombre_y_apellido, direccion)
VALUES (28999111, 'Mario Lopez', 'Lavalle 455');
INSERT INTO persona (dni, nombre_y_apellido, direccion)
VALUES (33444555, 'Pedro Suarez', 'Mitre 900');

INSERT INTO vehiculo (patente, marca, modelo, color, saldo_actual)
VALUES ('AA123BB', 'Renault', 2018, 'gris', 1500.00);
INSERT INTO vehiculo (patente, marca, modelo, color, saldo_actual)
VALUES ('AC456DD', 'Ford', 2020, 'negro', 800.00);
INSERT INTO vehiculo (patente, marca, modelo, color, saldo_actual)
VALUES ('AD789EE', 'Fiat', 2016, 'azul', 1200.00);

INSERT INTO parquemetro (id_parquemetro, calle, altura) VALUES (9, 'Italia', 1200);
INSERT INTO parquemetro (id_parquemetro, calle, altura) VALUES (10, 'Sarmiento', 845);
INSERT INTO parquemetro (id_parquemetro, calle, altura) VALUES (11, 'San Martin', 2300);

INSERT INTO duenio (patente, dni) VALUES ('AA123BB', 30111222);
INSERT INTO duenio (patente, dni) VALUES ('AC456DD', 28999111);
INSERT INTO duenio (patente, dni) VALUES ('AD789EE', 33444555);

INSERT INTO estacionamiento
(patente, id_parquemetro, fecha, saldo_inicio, saldo_final, hora_entrada, hora_salida)
VALUES ('AA123BB', 9, DATE '2024-08-02', 1500.00, 1350.00,
        TIMESTAMP '2024-08-02 08:00:00', TIMESTAMP '2024-08-02 09:30:00');

INSERT INTO estacionamiento
(patente, id_parquemetro, fecha, saldo_inicio, saldo_final, hora_entrada, hora_salida)
VALUES ('AC456DD', 9, DATE '2024-08-02', 800.00, 650.00,
        TIMESTAMP '2024-08-02 10:15:00', TIMESTAMP '2024-08-02 11:45:00');

INSERT INTO estacionamiento
(patente, id_parquemetro, fecha, saldo_inicio, saldo_final, hora_entrada, hora_salida)
VALUES ('AD789EE', 10, DATE '2024-08-03', 1200.00, 1000.00,
        TIMESTAMP '2024-08-03 18:00:00', TIMESTAMP '2024-08-03 19:40:00');

INSERT INTO estacionamiento
(patente, id_parquemetro, fecha, saldo_inicio, saldo_final, hora_entrada, hora_salida)
VALUES ('AA123BB', 11, DATE '2024-08-04', 1350.00, 1250.00,
        TIMESTAMP '2024-08-04 16:00:00', TIMESTAMP '2024-08-04 16:50:00');
```

Por qué funciona

- Oracle 10/11 no acepta `INSERT ... VALUES (...), (...)` con múltiples filas como MySQL/PostgreSQL: cada fila va en su propio INSERT.
- Las fechas y horas se escriben con los literales `DATE '...'` y `TIMESTAMP '...'` para que Oracle las parsee sin depender del `NLS_DATE_FORMAT` de la sesión.
- El `CHECK (LOWER(color) IN (...))` permite cargar el color en minúsculas tal como se hace aquí.

3.4. Inciso c) – MySQL/PostgreSQL

Solución en SQL

El mismo bloque de INSERT corre sin cambios en MySQL 8+ (practico1c_mysql) y en PostgreSQL 14+ (practico1c): la sintaxis multi-fila con VALUES (...), (...) está soportada por ambos.

```
INSERT INTO cliente (dni, nombre, apellido, direccion, tarifa) VALUES
(30111222, 'Lucas', 'Gomez', 'Colon 120', 3500),
(28999111, 'Martin', 'Sosa', 'Belgrano 455', 4200),
(33444555, 'Sergio', 'Perez', 'Rivadavia 900', 2800);

INSERT INTO categoria (nro_categoria, tasa) VALUES
(1, 0.80),
(2, 1.10),
(3, 1.35);

INSERT INTO taller (nro_taller, nombre, direccion) VALUES
(1, 'Taller Centro', 'Sobremonte 100'),
(2, 'Taller Sur', 'Ruta 8 km 601');

INSERT INTO automovil (patente, marca, modelo, dni, nro_categoria) VALUES
('AAA111', 'FIAT', 2010, 30111222, 1),
('BBB222', 'FORD', 2012, 30111222, 2),
('CCC333', 'RENAULT', 2015, 28999111, 1),
('DDD444', 'FIAT', 2008, 33444555, 3);
-- TODO: el enunciado pide modelo entre 1990 y 2015; 'DDD444' modelo 2008 cumple,
-- pero conviene revisar si todos los autos de prueba deben estar dentro del rango.

INSERT INTO accidente (nro_accidente, dni, patente, nro_taller, fecha, costo) VALUES
(5001, 30111222, 'AAA111', 1, '2024-01-10', 150000.00),
(5002, 30111222, 'BBB222', 2, '2024-03-12', 230000.00),
(5003, 30111222, 'AAA111', 1, '2024-06-01', 80000.00),
(5004, 30111222, 'BBB222', 1, '2024-07-20', 120000.00),
(5005, 28999111, 'CCC333', 2, '2024-05-05', 95000.00),
(5006, 33444555, 'DDD444', 1, '2024-08-09', 110000.00);
```

Por qué funciona

- Lucas Gómez (DNI 30111222) aparece con cuatro accidentes, lo que garantiza al menos una fila en la consulta 3.c (HAVING COUNT(*) >3).
- Todas las marcas (FIAT, FORD, RENAULT) y modelos respetan los dominios definidos, así que ningún INSERT es rechazado por los CHECK.
- Las tarifas (3500, 4200, 2800) están dentro del rango (0, 10000) pedido.

4. Ejercicio 3: Consultas SQL y álgebra relacional

Consigna

Utilizando las bases de datos creadas en los incisos del ejercicio 1, resolver las siguientes consultas:

Qué se pide / idea

Cinco consultas que cubren los temas centrales del DML: filtrado con WHERE, ordenamiento con ORDER BY, joins, agregación con GROUP BY/HAVING, subconsultas correlacionadas y, en el último

inciso, traducción entre álgebra relacional y SQL.¹

Portabilidad entre motores. Todas las queries de los incisos 3.a a 3.d están escritas en SQL estándar (SELECT, JOIN ... ON, WHERE, GROUP BY, HAVING, ORDER BY, DISTINCT, NOT EXISTS, MAX/MIN/COUNT) y corren sin cambios en MySQL 8+, PostgreSQL 14+ y Oracle 10/11 XE; por eso se muestra una sola versión por consulta. La única excepción aparece en 3.e Consulta 2, donde el operador algebraico $\cap$ se traduciría a INTERSECT: PostgreSQL y Oracle lo soportan desde siempre, pero MySQL recién lo agregó en 8.0.31; por compatibilidad se resuelve con JOIN + DISTINCT (equivalente semántico).

4.1. a) Clientes sin ventas (base del inciso 1.a)

Consigna

(bases de datos del inciso 1 a) Listar los clientes (todos sus datos) que no se le han realizado ninguna venta (clientes que no tienen ninguna factura asociada). Al listado ordenarlo por apellido y nombre en forma descendente.

Qué se pide / idea

Hay que devolver los clientes *huérfanos*: los que existen en **cliente** pero no aparecen en **factura**. El patrón canónico para esto es una subconsulta correlacionada con NOT EXISTS, que se lee literalmente como “no existe ninguna factura asociada a este cliente”.

Solución en SQL

```
SELECT c.nro_cliente,
       c.apellido,
       c.nombre,
       c.direccion,
       c.telefono
FROM cliente c
WHERE NOT EXISTS (
    SELECT 1
    FROM factura f
    WHERE f.nro_cliente = c.nro_cliente
)
ORDER BY c.apellido DESC, c.nombre DESC;
```

Por qué funciona

- El FROM **cliente c** recorre todos los clientes; el NOT EXISTS descarta los que tienen al menos una factura.
- Se prefiere NOT EXISTS a NOT IN (SELECT nro_cliente FROM factura) porque, si alguna factura tuviera nro_cliente NULL, NOT IN devolvería el conjunto vacío (consecuencia de la lógica trivaluada). NOT EXISTS es robusto frente a esos casos.
- El ORDER BY c.apellido DESC, c.nombre DESC aplica el desempate por nombre solo cuando dos clientes comparten apellido.
- Con los datos del Ejercicio 2 la consulta devuelve a Diego Luna, único cliente sin facturas.

¹En el PDF original, el segundo ítem del Ejercicio 3 figura etiquetado como “d)” cuando por contexto debería ser “b)”. Se conserva la numeración del PDF en la transcripción de cada consigna, pero se identifica acá como 3.b para que el índice quede legible.

4.2. b) Vehículos que usaron el parquímetro 9 (base del inciso 1.b)

Consigna

(bases de datos del inciso 1 b) Listar los vehículos que utilizaron el parquímetro 9, indicar modelo y color.

Qué se pide / idea

Es un join clásico entre la tabla de hechos (**estacionamiento**, donde están los registros de uso) y la tabla de entidades (**vehiculo**, donde están modelo y color), filtrando por `id_parquimetro = 9`. El detalle a cuidar es la deduplicación: un mismo vehículo puede haberse estacionado varias veces en el parquímetro 9.

Solución en SQL

```
SELECT DISTINCT v.patente,
               v.modelo,
               v.color
FROM vehiculo v
JOIN estacionamiento e
  ON e.patente = v.patente
WHERE e.id_parquimetro = 9;
```

Por qué funciona

- El JOIN cruza **vehiculo** con **estacionamiento** por la patente, conservando solo las filas con un estacionamiento real.
- El WHERE `e.id_parquimetro = 9` filtra antes de la deduplicación, así el join trabaja solo con los registros relevantes.
- El DISTINCT elimina filas duplicadas que aparecerían si un vehículo se estacionó más de una vez en el parquímetro 9.
- Se incluye la **patente** junto a modelo y color, para distinguir vehículos distintos que casualmente compartan ambas características.

4.3. c) Clientes con más de 3 accidentes (base del inciso 1.c)

Consigna

(bases de datos del inciso 1 c) Listar los clientes cuya cantidad de accidentes que tuvieron es superior a 3. Listar DNI, nombre y apellido.

Qué se pide / idea

Es un caso típico de agregación con filtro posterior: agrupar accidentes por cliente, contar y quedarse solo con los grupos cuyo conteo es mayor a 3. La distinción clave entre WHERE y HAVING aparece exactamente aquí: HAVING se aplica después del GROUP BY, sobre los grupos, no sobre las filas originales.

Solución en SQL

```
SELECT c.dni,
       c.nombre,
       c.apellido
FROM cliente c
JOIN accidente a
```

```
ON a.dni = c.dni
GROUP BY c.dni, c.nombre, c.apellido
HAVING COUNT(*) > 3;
```

Por qué funciona

- El JOIN encadena cada accidente con su cliente; el resultado intermedio tiene una fila por accidente.
- El GROUP BY c.dni, c.nombre, c.apellido colapsa todas las filas de un mismo cliente en un solo grupo. Se agrupa por las tres columnas porque en SQL estándar y en PostgreSQL todas las columnas no agregadas del SELECT deben aparecer en el GROUP BY.
- HAVING COUNT(*) >3 se aplica sobre los grupos ya armados: no se podría usar WHERE COUNT(*) >3 porque WHERE se evalúa antes de la agregación.
- Con los datos cargados, el único cliente que cumple es Lucas Gómez, que tiene cuatro accidentes.

4.4. d) Máximo y mínimo monto de factura por cliente (base del inciso 1.a)

Consigna

(bases de datos del inciso 1 a) Obtener el máximo y mínimo precio de montos pagados en las facturas por cada cliente. Listar Nro_cliente, nombre y máximo y mínimo monto de factura.

Qué se pide / idea

Otra agregación: por cada cliente, calcular MAX(monto) y MIN(monto) sobre sus facturas. El JOIN interno se encarga de descartar a los clientes sin facturas, para los que no tendría sentido un máximo o un mínimo.

Solución en SQL

```
SELECT c.nro_cliente,
       c.nombre,
       c.apellido,
       MAX(f.monto) AS monto_maximo,
       MIN(f.monto) AS monto_minimo
FROM cliente c
JOIN factura f
  ON f.nro_cliente = c.nro_cliente
GROUP BY c.nro_cliente, c.nombre, c.apellido
ORDER BY c.nro_cliente;
```

Por qué funciona

- El JOIN interno (cliente JOIN factura) se queda solo con los clientes que tienen al menos una factura; un LEFT JOIN dejaría a Diego Luna en el listado con NULL en ambas agregaciones.
- GROUP BY arma un grupo por cliente; MAX y MIN se calculan dentro de cada grupo.
- Se agrega c.apellido al SELECT y al GROUP BY para que el listado sea legible aunque el enunciado solo mencione “nombre”; al ser una columna funcionalmente dependiente del nro_cliente, incluirla no altera los grupos.
- Con los datos cargados, Andrés Pérez es el caso ilustrativo: máximo \$450.000 y mínimo \$106.000.

4.5. e) Tres consultas en álgebra relacional

Consigna

Proponer y resolver utilizando el álgebra relacional al menos 3 consultas sobre la base de datos que usted desee. Los siguientes operadores: **selección**, **proyección**, **unión**, **intersección** y **producto cartesiano**; deben participar en al menos una consulta.

Qué se pide / idea

Se elige como base el inciso 1.a porque sus relaciones son simples y permiten ilustrar cada operador en una expresión corta. Entre las tres consultas se cubren los cinco operadores pedidos: selección (σ), proyección (π), unión ($\cup$), intersección ($\cap$) y producto cartesiano ($\times$). Cada consulta se da primero en notación algebraica y después en SQL equivalente.

Consulta 1 — Productos con stock por debajo o igual al mínimo

Consigna

Listar los productos cuyo stock disponible está por debajo del mínimo, o exactamente en el mínimo (es decir, los que necesitan reposición).

Qué se pide / idea

Se construye con una unión de dos selecciones, una por cada condición ($<$ y $=$). Es una forma artificial de ilustrar $\cup$: en un caso real se escribiría con un único $\leq$.

Solución en SQL

Álgebra relacional:

```
pi_{cod_producto, descripcion}(\sigma_{cantidad < stock_minimo}(Producto))
union
pi_{cod_producto, descripcion}(\sigma_{cantidad = stock_minimo}(Producto))
```

SQL equivalente:

```
SELECT cod_producto, descripcion
FROM producto
WHERE cantidad < stock_minimo

UNION

SELECT cod_producto, descripcion
FROM producto
WHERE cantidad = stock_minimo;
```

Por qué funciona

- Cada subconsulta proyecta (`cod_producto`, `descripcion`) después de aplicar su selección.
- UNION (sin ALL) ya elimina duplicados, igual que el $\cup$ algebraico, que trabaja con conjuntos.
- Operadores usados: selección (σ), proyección (π) y unión ($\cup$).

Consulta 2 — Productos facturados con stock por debajo del máximo

Consigna

Listar los productos que aparecen en al menos una factura y al mismo tiempo todavía tienen capacidad de stock (es decir, su `cantidad` es menor al `stock_maximo`).

Qué se pide / idea

Se modela como intersección entre dos conjuntos de `cod_producto`: los que figuran en `ItemFactura` y los que cumplen `cantidad < stock_maximo` en `Producto`.

Solución en SQL

Álgebra relacional:

```
pi_{cod_producto}(ItemFactura)
interseccion
pi_{cod_producto}(\sigma_{cantidad < stock_maximo}(Producto))
```

SQL equivalente:

```
SELECT DISTINCT i.cod_producto
FROM item_factura i
JOIN producto p
  ON p.cod_producto = i.cod_producto
WHERE p.cantidad < p.stock_maximo;
```

Por qué funciona

- El JOIN cruza ítems y productos; el WHERE aplica la selección sobre el lado del catálogo.
- DISTINCT replica la semántica de conjuntos del operador de intersección: un producto facturado en varias filas aparece una sola vez.
- Equivale a `SELECT cod_producto FROM item_factura INTERSECT SELECT cod_producto FROM producto WHERE cantidad < stock_maximo` en motores que soportan INTERSECT (PostgreSQL sí, MySQL no hasta 8.0.31).
- Operadores usados: selección, proyección e intersección.

Consulta 3 — Pares cliente-factura vía producto cartesiano

Consigna

Listar el par (cliente, factura) para cada factura del sistema, reconstruido a partir del producto cartesiano `Cliente × Factura` y la selección posterior por igualdad de `nro_cliente`.

Qué se pide / idea

Es la definición “de manual” del join: producto cartesiano más selección. Sirve para mostrar cómo un JOIN ... ON de SQL se descompone en operadores algebraicos primitivos.

Solución en SQL

Álgebra relacional:

```
pi_{Cliente.nro_cliente, Cliente.apellido, Factura.nro_factura}
(
  \sigma_{Cliente.nro_cliente = Factura.nro_cliente}
```

```
(Cliente x Factura)
)
```

SQL equivalente:

```
SELECT c.nro_cliente,
       c.apellido,
       f.nro_factura
FROM cliente c
JOIN factura f
ON c.nro_cliente = f.nro_cliente;
```

Por qué funciona

- **Cliente × Factura** produce todas las combinaciones posibles; la σ posterior conserva solo las que coinciden por **nro_cliente**, que es exactamente lo que define un **INNER JOIN** natural por esa columna.
- La proyección final descarta columnas innecesarias y deja un listado limpio.
- Operadores usados: producto cartesiano ($\times$), selección y proyección.

5. Conclusión

El práctico repasó las construcciones más usadas de SQL: DDL con restricciones (PK, UNIQUE, FK, CHECK, acciones referenciales), DML para cargar datos coherentes con el modelo y consultas que cubren desde filtros simples hasta agregaciones, subconsultas correlacionadas y la traducción de álgebra relacional a SQL.

También obligó a confrontar diferencias prácticas entre motores: cómo emular dominios cuando no existe **CREATE DOMAIN** (MySQL/Oracle), cómo implementar autonumeración en Oracle 10/11 XE con secuencias y triggers, y cuándo conviene cada motor para qué tipo de inciso. Las cargas del Ejercicio 2 se diseñaron específicamente para que las consultas del Ejercicio 3 devuelvan resultados visibles, lo que permite verificar tanto la corrección de las restricciones como la de las queries.